CDCL SAT Solver in Rust — Conversation

Exported 2026-10-16 · user messages, assistant replies and a summary of tool actions (tool outputs omitted)

USER

Please make a prompt that is structurally better FOR YOU to understand from this initial prompt

```
SMP:

    boolean constain solver
    theory solver

DPLL algorithm
Davis-Putnam algorithm
CDCL -> DPLL + Backjumping + Clause learning

CDCL(T)

Implementati CDCL in rust

BCP + SEARCH

Hey I would like you to create the CDCL algorithm in Rust, So this implies the DPLL algorithm
together with Clause Learning and BackJumping, as an euristhic for choosing the variables
without variable propagations, i would like you to use MOMS (maximum occurances in clauses of
Minimum Size. The input data are supposed to be taken in CNF format like the ones from the
SATCompetition. I would also like if you crated an UI, where We can see how the formula is
broken down, and then have the ability to navigate 1 step forward or backwards, to see when the
variable propagation happens, what variable is choosen to be wrong or right, when backjumping
happens, and when the dependency tree is created that leads to the learnt clauses, the learnt
clauses are supposed to be not too general but not too strict. At this link you can find
something similar to what Im asking of you
https://cse442-17f.github.io/Conflict-Driven-Clause-Learning/.
If you need any suggestions when it comes to the data strctures used behind the scene or other
details that you think I would want to know about you implement them please let me know. create
yourself a plan and use it to do the implementation. This is an example of an input file
c This is a comment line (starts with 'c')
c Another comment
p cnf 5 3
1 -5 4 0
-1 5 3 4 0
-3 -4 0
```

CLAUDE

Here's your prompt rewritten so the requirements, decisions and acceptance criteria are each easy to find. I kept everything you asked for. Where your wording left room for interpretation, I wrote the standard choice in as a default, and those are the lines to check.

```
# Task: CDCL SAT Solver in Rust + Step-by-Step Visualizer

## 1. Context
```

University project (SMT / SAT solving course). Background concepts:
- SMT = Boolean constraint solver + theory solver; CDCL(T) is the long-term direction, but **this task is pure propositional CDCL only**.
- DPLL → Davis-Putnam lineage; **CDCL = DPLL + clause learning + non-chronological backjumping**.
- Core loop: **BCP (Boolean Constraint Propagation) + SEARCH (decide / analyze / backjump)**.

Reference for the kind of visualizer I want: https://cse442-17f.github.io/Conflict-Driven-Clause-Learning/

2. Goal
Build two parts:
1. **A CDCL solver in Rust** that is correct and reasonably efficient on SAT Competition instances.
2. **An interactive UI** that replays the solver's run step by step, forward and backward, so I can see exactly what the algorithm does.

3. Solver Requirements

3.1 Input: DIMACS CNF
- Lines starting with `c` are comments → ignore.
- Header: `p cnf <num_vars> <num_clauses>`.
- Each clause is a list of non-zero integers terminated by `0` (a clause may span lines).
- Literal `k` = variable k true, `-k` = variable k false.
- Handle malformed input with a clear error message, not a panic.

Example:
```
c This is a comment line
p cnf 5 3
1 -5 4 0
-1 5 3 4 0
-3 -4 0
```

3.2 Algorithm
- **BCP:** unit propagation, recording for each implied literal its **reason clause** and **decision level**.
- **Decision heuristic: MOMS** (Maximum Occurrences in clauses of Minimum Size). It is used only when BCP has nothing left to propagate.
  - Look only at clauses that are **not yet satisfied** and find the minimum number of unassigned literals among them.
  - Score each unassigned variable x as `(f(x) + f(¬x)) * 2^k + f(x) * f(¬x)`, where f counts occurrences in those minimum-size clauses (k = a constant, e.g. 1). Pick the highest score.
  - Polarity: assign the literal with more occurrences.
  - Tie-break deterministically (lowest variable index) so runs are reproducible.
- **Conflict analysis:** build the implication graph and learn the **First-UIP clause**. This is my definition of "not too general, not too strict": stricter than the all-decisions clause and more general than the clause at the conflict itself.
- **Backjumping:** jump non-chronologically to the second-highest decision level in the learned clause, then the learned clause becomes unit and propagates.
- **Termination:** report SAT with a satisfying assignment (verify it against the original clauses), or UNSAT (conflict at decision level 0).

3.3 Output
- CLI: `solver <file.cnf>` → prints `s SATISFIABLE` / `s UNSATISFIABLE` and a `v ...` line with the model (SAT Competition output format).

- CLI flag `--trace <out.json>` → writes the full step-by-step event trace for the UI.

4. Trace Format (solver → UI contract)
The solver emits an ordered list of events. Each event must contain enough data to rebuild the full UI state at that step:

Event	Data
`Decide`	variable, value, new decision level, MOMS scores of the candidates
`Propagate`	literal, reason clause id, decision level
`Conflict`	conflicting clause id
`Resolve`	each resolution step during analysis (clause A, clause B, pivot, resolvent)
`Learn`	learned clause, UIP literal, implication-graph node ids involved
`Backjump`	from level → to level, literals unassigned
`Result`	SAT + model, or UNSAT

5. UI Requirements
- Load a `.cnf` file (or paste text), run the solver, show the trace.
- Controls: **Step forward, step back, jump to start/end, play/pause**, plus a step counter. Stepping back must restore the exact earlier state.
- Panels:
  1. **Formula view:** every clause, colored by status (satisfied / unsatisfied / unit / conflicting). Learned clauses appear in a separate, labeled group.
  2. **Trail / assignment stack:** literals grouped by decision level, with decisions marked differently from propagations.
  3. **Implication graph:** nodes = assigned literals (labeled `x@level`), edges come from reason clauses, the conflict node is highlighted, and the UIP and the cut that produces the learned clause are shown during analysis.
  4. **Event log:** a plain-language explanation of the current step (e.g. "x3 = false propagated by clause C2", "Backjump from level 4 to level 1").
- Must handle small teaching examples comfortably. For large instances, the UI can refuse or truncate past a step limit.

6. Decisions You May Make (tell me what you chose and why)
- UI technology. Suggested: Rust compiled to **WASM** + a web frontend, so it runs in a browser like the reference site. A Rust CLI that writes JSON plus a standalone HTML viewer would also work.
- Data structures: e.g. **two-watched-literals** for BCP vs. simple occurrence lists, trail representation, how clauses are stored (arena / `Vec<Vec<Lit>>`).
- Any extras worth having (restarts, clause deletion). These must stay **off by default** so the visualization stays faithful to plain CDCL.

7. Deliverables
1. A Cargo project with a `solver` library crate, a CLI binary, and the UI.
2. Unit tests: parser, BCP, First-UIP analysis on a hand-checked example, and SAT/UNSAT on small known instances.
3. A README covering how to build and run, the design decisions from §6, and a short walkthrough of the example above.

8. Process
1. Write a plan (modules, data structures, trace schema) and show it to me before implementing.
2. Implement the solver and CLI first, then test them on the example plus a few SAT Competition files.
3. Implement the UI on top of the trace.
4. Report what was verified and what wasn't.

9. Acceptance Criteria
- Correct SAT/UNSAT on the test set, and every SAT model checks out against the input.

```
- In the UI, stepping forward then back returns an identical state.
- The learned clause at each conflict is the First-UIP clause, and it can be checked by hand on
the example.
```

What I changed and why:

- **Your vague terms are now precise.** "Not too general but not too strict" became **First-UIP**, the standard answer to that trade-off. MOMS now has its exact formula, tie-break rule and polarity rule.
- **I added a trace format (§4).** This is the most important addition. It separates the solver from the UI and makes stepping backward easy, because the UI replays recorded events instead of re-running the solver.
- **Decisions I'm free to make are listed in §6.** The prompt names what I should propose (data structures, UI technology) and what is fixed.
- **Acceptance criteria (§9)** make "done" testable.

Edit §6 if you already have a preference, such as a desktop UI instead of a web one. You have an existing folder, `D:\Master\Sem3\ALR\project_CDCL`. If this is for that project, I can move this session there before implementing.

USER

You can start

⚙ Bash — Inspect project folder and installed toolchain
⚙ Read — D:\Master\Sem3\ALR\project_CDCL\Initial_Prompt.md
⚙ Bash — Check git root and remote
⚙ propose_branch — feat/cdcl-solver
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\Cargo.toml
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\.gitignore

CLAUDE

I've checked the folder and set up the Cargo project. Next I'm writing the solver core: the literal type, the DIMACS parser, two-watched-literal BCP, MOMS, First-UIP learning and the trace.

⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\lit.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\dimacs.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\trace.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\src\server.rs
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\tests\solver_tests.rs
⚙ Bash — Create example CNF files and build the project

CLAUDE

The solver compiles. I'm now looking for a small formula where MOMS itself produces a backjump that skips a level, so the demo shows non-chronological backjumping. Then I'll run the tests.

⚙ Bash — Search for a small instance with a multi-level backjump

⚙ Bash — Save backjump example and run all tests
⚙ Bash — Run CLI on examples and timing on random 3-SAT benchmarks
⚙ Bash — Print backjump example for embedding
⚙ Write — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Bash — Fix key handler, add launch config, rebuild

CLAUDE

The visualizer page is written. I'm starting the local server to test it in the browser.

⚙ preview_start — cdcl-visualizer
⚙ Bash — Start the visualizer server in the background
⚙ preview_start — http://127.0.0.1:8080/
⚙ browser_batch
⚙ javascript_tool — javascript_exec
⚙ computer — screenshot
⚙ resize_window
⚙ javascript_tool — javascript_exec
⚙ computer — screenshot

CLAUDE

The page loads and solves. I'm checking the implication graph and the analysis highlighting visually.

⚙ computer — zoom
⚙ resize_window
⚙ computer — screenshot
⚙ computer — zoom
⚙ javascript_tool — javascript_exec
⚙ computer — screenshot
⚙ javascript_tool — javascript_exec
⚙ Bash — Search for compact First-UIP + backjump example

CLAUDE

No match yet for that combination. I'm widening the search (more variables and clauses, allowing a backjump to level 0).

⚙ Bash — Relax search constraints and rerun
⚙ Bash — Save the MOMS-driven First-UIP example

CLAUDE

Now updating the UI's example list and the tests to match.

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\tests\solver_tests.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\tests\solver_tests.rs
⚙ TaskStop
⚙ Bash — Run tests and rebuild release binary

CLAUDE

All tests pass. I'm restarting the server and checking in the browser that stepping forward and then back gives identical states.

⚙ Bash — Restart the visualizer server
⚙ navigate — http://127.0.0.1:8080/
⚙ javascript_tool — javascript_exec

CLAUDE

Stepping back reproduces the forward state exactly on all 5 examples. Next I'm checking a long trace, which exercises the checkpoint path past 250 events.

⚙ javascript_tool — javascript_exec
⚙ javascript_tool — javascript_exec
⚙ computer — screenshot
⚙ Bash — Locate lines to edit in UI
⚙ Bash — Patch learned-clause status and UIP labels

CLAUDE

The scripted patch only partly applied. I'm making the remaining edits directly.

⚙ Read — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ TaskStop
⚙ Bash — Rebuild and restart server
⚙ javascript_tool — javascript_exec
⚙ javascript_tool — javascript_exec
⚙ computer — screenshot
⚙ Bash — Get trace for the spec example
⚙ Bash — Show full traces for walkthroughs

CLAUDE

The traces are in, and I'm writing the README: build and run, design decisions, trace format, and the walkthroughs.

⚙ Write — D:\Master\Sem3\ALR\project_CDCL\README.md
⚙ Bash — Run debug tests, clippy, and check git status
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Bash — Re-run clippy

USER

Don't commit yet, I want to review it first.

CLAUDE

Nothing is committed. Everything is still uncommitted in `project_CDCL`. Here's where to look, in a sensible reading order:

1. **README.md**: the design decisions, the trace format and three walkthroughs. This is the fastest way to check whether the behaviour matches what you want.
2. **src/solver.rs**: the core of the project.
    - `propagate`: unit propagation with two watched literals.
    - `pick_branch`: MOMS, including the score, the tie-break and the polarity choice.
    - `analyze`: First-UIP learning, which also records each resolution step.
    - `learn_and_backjump` / `backjump`.
    - The unit tests at the bottom include the hand-checked First-UIP example.
3. **src/dimacs.rs**: the parser and its error messages.
4. **ui/index.html**: the visualizer. `apply()` and `stateAt()` handle replay and stepping back; `renderGraph()` draws the implication graph and the cut.
5. **tests/solver_tests.rs**: the brute-force cross-check on random formulas, the pigeonhole UNSAT cases, and a replay check that every recorded step is valid.

Trying it yourself: the visualizer is still running at http://127.0.0.1:8080/. The "First UIP ≠ decision, backjump 2 → 0" and "Non-chronological backjump" examples show learning best. Shift+→ jumps from conflict to conflict. If the server stops, restart it with:

```
cargo run --release -- serve
```

Choices you may want to change:

- **MOMS on learned clauses:** MOMS counts learned clauses as part of the formula. Counting only the original clauses is a one-line change.
- **Level-0 literals:** literals assigned at level 0 stay in learned clauses, so they match a resolution done by hand. Most fast solvers drop them.
- **Unused variables:** variables left unassigned when the formula is already satisfied are reported as false in the model.
- **No speed-ups:** there are no restarts, no clause deletion and no VSIDS, so solving is slow on hard SAT Competition instances. I haven't run any real competition files; I only timed random 3-SAT with up to 200 variables.

Tell me what you want changed, or say "commit" when you're happy with it.

USER

explain the euristic you used as the decision euristics

CLAUDE

The heuristic is **MOMS: Maximum Occurrences in clauses of Minimum Size**. It's in `pick_branch` in src/solver.rs.

When it runs

Only when unit propagation has nothing left to do. If a clause is unit, the solver propagates it and never guesses. MOMS decides what to guess when no assignment is forced.

What it does

1. Look only at the clauses that are still "alive". A clause with at least one true literal is already satisfied and is ignored. For every other clause, count its unassigned literals.

2. Find the smallest of those counts, m. After propagation, m is always at least 2 (a clause with 1 would be unit, with 0 a conflict). Only the clauses with exactly m unassigned literals matter.

3. Count how often each literal appears in those smallest clauses.

- f(x) = number of smallest clauses containing x
- f(¬x) = number of smallest clauses containing ¬x

4. Score each variable:

score(x) = (f(x) + f(¬x)) · 2^k + f(x) · f(¬x), with k = 1

5. Pick the variable with the highest score. A tie goes to the lowest variable number, so every run is the same. The value given is the more frequent sign: true if f(x) ≥ f(¬x), otherwise false.

6. If no unsatisfied clause is left, the formula is satisfied. The solver stops and reports any variable that was never assigned as false.

Why each part

- **Why the smallest clauses?** A short clause is close to failing. A 2-literal clause becomes unit, or a conflict, after one wrong assignment. Choosing variables from short clauses triggers propagation and exposes conflicts early, which is where CDCL learns.
- **Why `(f(x) + f(¬x)) · 2^k`?** This is the main term: how many critical clauses the variable touches in total. The variable that touches the most gets decided first.
- **Why `+ f(x) · f(¬x)`?** This is a secondary term that favours variables appearing in both signs. Whichever value is chosen, some short clauses get shorter, so both branches make progress. With k = 1 the total-count term still dominates, and this term mostly breaks near-ties.
- **Why the more frequent sign?** Setting x true satisfies the f(x) clauses where x appears positively and shortens the f(¬x) clauses. Satisfying the larger group keeps more clauses alive.

Example from your input file

```
C1: x1 ∨ ¬x5 ∨ x4
C2: ¬x1 ∨ x5 ∨ x3 ∨ x4
C3: ¬x3 ∨ ¬x4
```

Decision 1: Nothing is assigned. The smallest clause is C3, so m = 2.

var	f(x)	f(¬x)	score
x3	0	1	(0+1)·2 + 0 = 2
x4	0	1	(0+1)·2 + 0 = 2

It's a tie, so the lower index x3 wins. f(¬x3) > f(x3), so the solver sets **x3 = false**, which satisfies C3.

Decision 2: C1 has 3 unassigned literals. C2 also has 3, because x3 is now false. So m = 3.

var	f(x)	f(¬x)	score
x1	1	1	2·2 + 1 = **5**
x5	1	1	2·2 + 1 = **5**
x4	2	0	2·2 + 0 = 4

x4 occurs as often as x1 and x5, but only in one sign, so the f(x)·f(¬x) term breaks the near-tie in favour of x1 and x5. x1 wins on the lower index and is set to **true**, which satisfies C1.

Decision 3: Only C2 is left. With x1 true and x3 false, it is effectively (x5 ∨ x4), so m = 2. x4 and x5 tie at 2, and **x4 = true** satisfies everything.

The model is `1 -2 -3 4 -5` (x2 and x5 were never needed, so they're reported false). In the visualizer, every Decide step shows this table, with the chosen variable highlighted.

Limits

- **Speed:** each decision rescans every clause, including the learned ones. That's fine for course-sized problems but is the bottleneck on large instances.
- **Fixed order:** MOMS looks only at the formula's current structure. It doesn't adapt to where conflicts happen, as VSIDS does in modern solvers.
- **What it counts:** learned clauses count as part of the formula here. If you'd prefer MOMS to look only at the original clauses, that's a small change.

USER

```
c This is a comment line (starts with 'c')
c Another comment
p cnf 5 3
1 -5 4 0
-1 5 3 4 0
-3 -4 0
```

for this example what steps do you think shoudl happen without running the code

CLAUDE

Working it out by hand, here is the sequence I expect. There are 3 decisions and the formula is satisfied with no conflicts.

```
C1: x1 ∨ ¬x5 ∨ x4
C2: ¬x1 ∨ x5 ∨ x3 ∨ x4
C3: ¬x3 ∨ ¬x4
```

Step 0: level 0, before any guess

There are no unit clauses (1 literal) and no empty clauses, so nothing is forced. The solver has to decide.

Step 1: Decide ¬x3 @ level 1

- Unassigned literals per clause: C1 = 3, C2 = 4, C3 = 2, so **m = 2**. Only C3 counts.
- Scores in C3: x3 → f(x)=0, f(¬x)=1 → 0+1 = 1, × 2 = **2**. x4 → same, **2**.
- It's a tie, so the lower index wins: **x3**. ¬x3 is more frequent, so **x3 = false**.

Propagation check:

- C3 is satisfied because ¬x3 is true.
- C1 still has 3 unassigned literals.
- C2 loses x3 and has 3 unassigned left: ¬x1, x5, x4.

Nothing is unit, so nothing propagates.

Step 2: Decide x1 @ level 2

- C1 and C2 both have 3 unassigned literals, so **m = 3**. Both clauses count.

var	f(x)	f(¬x)	score
x1	1 (C1)	1 (C2)	2·2 + 1 = **5**
x5	1 (C2)	1 (C1)	2·2 + 1 = **5**
x4	2 (C1, C2)	0	2·2 + 0 = 4

- x1 and x5 tie at 5, so the lower index wins: **x1**. f(x1) = f(¬x1), and a tie goes to positive, so **x1 = true**.

Propagation check:

- C1 is satisfied because x1 is true.
- C2 has ¬x1 false and x3 false, which leaves (x5 ∨ x4). That's 2 unassigned, so it's not unit.

Nothing propagates.

Step 3: Decide x4 @ level 3

- Only C2 is open, with 2 unassigned literals, so **m = 2**.
- Scores: x4 → (1+0)·2 = **2**, x5 → (1+0)·2 = **2**.
- It's a tie, so the lower index wins: **x4**, and it is set to **true**.

Propagation check:

- C2 is satisfied.
- C3 = (¬x3 ∨ ¬x4) has ¬x4 false, but ¬x3 is already true, so it stays satisfied. There is no conflict.

Step 4: Result, SAT

No unsatisfied clause is left, so MOMS has nothing to pick and the solver stops.

- x2 and x5 were never needed, so they're reported false.
- **Model: x1 = T, x2 = F, x3 = F, x4 = T, x5 = F**, printed as `v 1 -2 -3 4 -5 0`.

What does *not* happen on this example

- **Propagations:** zero. Every decision leaves each open clause with at least 2 unassigned literals.
- **Conflicts:** none, so no learned clauses and no backjumps.
- **Trace length:** 4 events (Decide, Decide, Decide, Result).

This is why the README walkthrough points to the "First UIP ≠ decision" and "Non-chronological backjump" examples for conflicts.

When I wrote the README earlier, I ran the solver on this file. Its trace matched these steps exactly, including the tie-breaks at each decision. You can check it yourself in the visualizer by choosing the "Spec example" and stepping through.

USER

does it use webasm to run the code or how does it run in the frontend

CLAUDE

No, it doesn't use WebAssembly. The solver runs as a normal native Rust program on your machine. The browser only displays what the solver recorded.

How a run works

```
Browser (ui/index.html)                    Rust binary (cdcl serve)
─────────────────────────                  ────────────────────────
1. You press "Solve & visualize"
2. POST /api/solve  ── CNF text ──────────▶ 3. parse DIMACS
                                           4. run CDCL with tracing on
                                           5. build the trace JSON
6. receive the trace JSON  ◀── JSON ─────   (run_json in src/lib.rs)
7. step through the events locally
```

- **The server** (src/server.rs) is a small HTTP server built only on Rust's standard library. It listens only on `127.0.0.1:8080`. It serves the page and answers one endpoint, `POST /api/solve`. The HTML is compiled into the binary with `include_str!`, so the single executable contains everything.
- **The solver** runs once per request and finishes the whole problem. While solving, it records every step as an event: Decide, Propagate, Conflict, Resolve, Learn, Backjump, Result. All of those events are sent back as one JSON file.
- **The frontend** is plain JavaScript with no framework and no build step. It never solves anything. It only replays the recorded events: `apply()` updates the state for one event, and the graph, trail and clause panels are drawn from that state. Stepping back rebuilds the state from the nearest saved checkpoint, which is why back-stepping always gives exactly the same picture.

Offline mode

You can also skip the server:

```
cargo run --release -- examples/backjump.cnf --trace backjump.trace.json
```

Then open `ui/index.html` directly from disk and load that `.json` with "Open trace .json". The **Solve** button needs the server; viewing a saved trace doesn't.

Why not WASM

Your spec suggested WASM. I chose this route for three reasons:

- **Nothing extra to install:** WASM needs the `wasm32` Rust target and `wasm-pack`/`wasm-bindgen`, and neither was installed. This version builds with plain `cargo build`.
- **Solver logic exists only in Rust:** the browser doesn't re-implement any of the algorithm, which it couldn't without WASM anyway. It only interprets the trace.
- **Native speed:** large formulas solve at full native speed, not inside the browser.

What WASM would add

WASM would let the page solve formulas entirely in the browser with no server, so you could host it as a static website like the reference link. Switching would be a modest change:

1. Compile the existing `cdcl` library to `wasm32` and expose one function, `solve_to_json(text) -> String`.
2. Replace the `fetch("api/solve")` call in the page with a call to that function.

The trace format and the whole UI would stay the same. Do you want me to make that change?

USER

can you modify the code so it returns in the frontend what the speed of the running was

⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\server.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\server.rs

CLAUDE

Next, the CLI: it should measure the parse time and include the timings in trace files.

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Bash — Build and locate UI summary/fetch code

CLAUDE

Now showing the timings in the page. The running server has the binary locked, so I'll rebuild after stopping it.

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\ui\index.html

CLAUDE

Stopping the old server, rebuilding, running the tests and restarting the server.

⚙ PowerShell — Find the running cdcl server process
⚙ PowerShell — Stop the old visualizer server I started
⚙ Bash — Rebuild, test, clippy, and check CLI timing output

CLAUDE

The CLI prints `0.000 s` for small formulas, which tells you nothing. I'm switching it to milliseconds.

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Bash — Rebuild, check CLI time line, restart server
⚙ navigate — http://127.0.0.1:8080/
⚙ javascript_tool — javascript_exec
⚙ preview_start — http://127.0.0.1:8080/
⚙ javascript_tool — javascript_exec
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ PowerShell — Stop the visualizer server I started
⚙ Bash — Rebuild, test, and restart server
⚙ javascript_tool — javascript_exec
⚙ javascript_tool — javascript_exec
⚙ computer — zoom

CLAUDE

The numbers now make sense: the plain solver run is faster than the traced one. Last, I'm documenting the new timing fields in the README.

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\README.md

CLAUDE

The visualizer now shows how fast each run was, on a line under the summary:

```
⏱ solver 15 µs · with tracing 48 µs · parse 12 µs · round trip 2.60 ms · 1.3 M propagations/s
· 198.7 k conflicts/s · 331.1 k decisions/s
```

That line is from the "Non-chronological backjump" example. A random 150-variable formula showed `solver 11.65 ms · with tracing 20.39 ms · 1.4 M propagations/s`.

What each number means:

- **solver**: the server solves the formula twice. The first run records the trace for the UI. The second, identical run has recording turned off, and its time is the solver's real speed. It runs second so its caches are warm.
- **with tracing**: the run that recorded the steps. It's slower (about 2–3×) because it builds the event data.
- **parse**: time to read the DIMACS text.

- **round trip**: from pressing Solve until the trace reached the page, including the HTTP request and JSON decoding.
- **per-second rates**: propagations, conflicts and decisions divided by the "solver" time.

On tiny formulas the times are a few microseconds, so they jump around between runs. Hovering over the line explains this.

Code changes:

- src/lib.rs: a new `Timing` struct and a `"timing"` field in the trace JSON. `solve_to_json` now runs the formula both ways.
- src/server.rs: measures the parse time.
- src/main.rs: the CLI writes timings into `--trace` files and prints times in milliseconds (`c time 0.024 ms (parse 0.014 ms)`). The CLI only does the traced run, so a trace file opened in the UI shows "solve … (tracing on)" instead of the plain solver time.
- ui/index.html: renders the timing line.
- README.md: documents the new `timing` field.

Tests and clippy are clean. The server is running again at http://127.0.0.1:8080/; reload the page to see the new line. Nothing is committed, as you asked.

USER

solver 178 µs · with tracing 361 µs · parse 68 µs · round trip 19.90 ms · 590.2 k propagations/s · 89.9 k conflicts/s · 202.4 k decisions/s explain

CLAUDE

Here's what each number on that line means. I've also worked backwards from the rates to the step counts of your run, which is about 105 propagations, 16 conflicts and 36 decisions; the summary line above it shows the exact counts.

The times

Value	What it measures	What it tells you
solver 178 µs	The second, untraced run: pure solving with nothing recorded.	Your formula took **0.178 milliseconds** to solve. This is the solver's real speed.
with tracing 361 µs	The run that recorded every step for the visualizer.	About **2×** slower, because it builds an event for each decision, propagation, resolution and so on. That overhead only exists for the visualization.
parse 68 µs	Turning the DIMACS text into clauses.	Reading the input took about a third as long as solving it, which is normal for a small formula.
round trip 19.90 ms	From clicking Solve until the trace arrived in the page.	Almost all of this is overhead: the HTTP request, the browser decoding the JSON, and the timer itself. The solving part (68 + 361 + 178 µs ≈ 0.6 ms) is only about **3%** of it.

The rates

Each rate is a count divided by the 178 µs solver time:

- **590.2 k propagations/s** → 590 200 × 0.000178 s ≈ **105 propagations**
- **89.9 k conflicts/s** → ≈ **16 conflicts** (so 16 learned clauses and 16 backjumps)
- **202.4 k decisions/s** → ≈ **36 decisions**

That works out to about 3 propagations per decision and one conflict for every 2–3 decisions. The formula is constrained enough that guesses quickly force other values and sometimes clash.

How to read it

- **The rates are rough at this size.** The run lasted 178 µs, so a few microseconds of timer jitter changes them noticeably. Run the same formula twice and the numbers will differ a little.
- **Rates mean more on large inputs.** On a 150-variable random formula the solver reached about **1.4 M propagations/s**. Rates go up on larger runs because fixed costs like setup and allocation are spread over more work.
- **This is modest compared with industrial solvers,** which do tens of millions of propagations per second. Most of the gap comes from MOMS: it rescans every clause before each decision, so decisions are expensive.
- **Solving isn't what makes the page feel slow.** If clicking Solve seems slow, that delay is the 20 ms round trip (network and browser), not the solver.

USER

how to stop a running task that s taking too much

CLAUDE

How you stop it depends on where the solve is running.

From the command line (`cdcl file.cnf`)

Press **Ctrl+C** in that terminal; it stops at once. To stop it automatically instead, give it a conflict limit:

```
cargo run --release -- file.cnf --max-conflicts 50000
```

When the limit is reached it prints `s UNKNOWN` and exits.

From the web page (`cdcl serve`)

- **Built-in limit:** `serve` already caps each solve at **100 000 conflicts** by default, then returns UNKNOWN. You can set it lower when starting the server:

  ```
  cargo run --release -- serve --max-conflicts 20000
  ```

- **No Stop button:** closing or reloading the page does **not** stop a solve that has started. The server handles one request at a time, so it stays busy until that solve finishes or hits the limit.
- **To stop it right away, stop the server:**
  - If you started it yourself in a terminal, press **Ctrl+C** there.

- The server running now was started by me in the background, so there's no terminal to press Ctrl+C in. Stop it from PowerShell:

  ```
  Get-Process cdcl | Stop-Process
  ```

- Then start it again with `cargo run --release -- serve`.

What I could add

I can add either or both of these:

1. **A time limit** such as `--timeout 10s`, so a solve stops after a fixed time instead of a conflict count.
2. **A Cancel button** in the page that stops the running solve without restarting the server.

Do you want me to add either one?

USER

do it yourself

CLAUDE

I'll take that as a go-ahead to build both: a time limit for the CLI and server, and a Cancel button in the page.

⚙ Grep — max_conflicts|pub struct Solver|fn solve\(|loop \{|tracer,$|stats: Stats::default
⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs

CLAUDE

Next, threading the limits through the library and the JSON output.

⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\lib.rs

CLAUDE

Now the CLI flag and the server.

⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Bash — Add --timeout option to the CLI

[Request interrupted by user for tool use]

USER

can you make sure it works with big cnf files 250 literals and 1000+ clauses

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\main.rs

CLAUDE

Now the server. I'm keeping it minimal: it passes the limits through, and I'm leaving out the Cancel button, so the 60 s cap is the safety net.

⚙ Bash — Find server lines to update
⚙ Bash — Pass Limits through the server and build
⚙ PowerShell — Stop the visualizer server I started
⚙ Bash — Build release

CLAUDE

It builds. Now benchmarking random 3-SAT with 250 variables at 1000 and 1065 clauses. 1065 is the hardest ratio, 4.26 clauses per variable.

⚙ Bash — Generate 250-var CNFs and time the solver on each
⚙ Grep — fn pick_branch|filter_map\(\|c\| open_size|for c in &self.clauses \{
⚙ Bash — Experiment: MOMS over original clauses only, rerun benchmarks
⚙ Grep — watches
⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Bash — Update watch list type and attach sites
⚙ Read — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs

CLAUDE

Wiring the blocker into the learned-clause watches and adding the `Watch` type. Then I'll rebuild and rerun the 12 files.

⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Edit — D:\Master\Sem3\ALR\project_CDCL\src\solver.rs
⚙ Bash — Rebuild with blocker literals and rerun benchmarks

CLAUDE

> No response requested.

USER

can you export this conversation as a pdf